

Transforming HoC Census and other CSV files from Long to Wide Format

Philip Curran

Using Python program `longto__wide`

The purpose of this program is to transform file in long format into wide format for uploading into the Postgres database **genelects**.

There are two components to successfully using this program:

- Create a YAML configuration file with the information about the input and resulting output CSV files and information about the fields within the file which are to be transformed from row format to column format.
- Run the program `longto__wide.py` with a suitable Python virtualenv. The environment must have the Python package Pandas installed and have the Python module `transform.py` in the Python path.

YAML file `transform.json`

One of these has to be produced for each original CSV file to be transformed. It should also be on the search path for Python.

The general structure of the YAML configuration file is as follows:

```
{ "input" : "/path/to/long/format/source/csv/file",
  "output" : "/path/to/wide/format/target/file/",
  "primary_key" : "<identifier field in source>",
  "static_cols" : [ "list", "of", "columns",
                    "to", "appear", "in", "target" ],
  "transform_cols" : [
    { "long_column" : "<fieldA>",
      "long_values" : "<fieldB>" },
    { "long_column" : "<fieldC>",
      "long_values" : "<fieldD>" }
    ...
  ]
}
```

The `primarykey` field must appear in the list of `staticcols` shown above. The `transformcols` list of pairs of field names indicate the names of fields which are in long format, i.e. whose values will become new columns in the resulting target file, together with the field to be used to provide the value for that new column. The list of pairs can be repeated as many times as required by the format of the original CSV source file.

A Quick Example

The household tenure data is a long file (an excerpt is shown below) with the field variables containing the names of properties and the two fields `Connum` and `Conpc` containing the values that relate to those properties.

```
(TDA) philc@colt:~/dataman$ csvlook --max-rows 20 /nfs/workspace/ge_data/census/household_tenure.csv
| ONSConstID | ConstituencyName | RegNationID | RegNationName | NatComparator | variable |
| ----- | ----- | ----- | ----- | ----- | ----- |
| E14001063 | Aldershot | E12000008 | South East | UK | Owns d |
| E14001063 | Aldershot | E12000008 | South East | UK | Owns w |
| E14001063 | Aldershot | E12000008 | South East | UK | Shared |
| E14001063 | Aldershot | E12000008 | South East | UK | All so |
| E14001063 | Aldershot | E12000008 | South East | UK | Privat |
| E14001063 | Aldershot | E12000008 | South East | UK | Other |
| E14001063 | Aldershot | E12000008 | South East | UK | Lives |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | Owns d |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | Owns w |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | Shared |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | All so |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | Privat |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | Other |
| E14001064 | Aldridge-Brownhills | E12000005 | West Midlands | UK | Lives |
| E14001065 | Altrincham & Sale West | E12000002 | North West | UK | Owns d |
| E14001065 | Altrincham & Sale West | E12000002 | North West | UK | Owns w |
| E14001065 | Altrincham & Sale West | E12000002 | North West | UK | Shared |
| E14001065 | Altrincham & Sale West | E12000002 | North West | UK | All so |
| E14001065 | Altrincham & Sale West | E12000002 | North West | UK | Privat |
| E14001065 | Altrincham & Sale West | E12000002 | North West | UK | Other
```

What would be desirable would be to have a wide format file with fields for each of the categories contained in variables for both the raw numbers (in field `Connum`) and the percentage of the uk population (in field `Conpc`).

We can achieve this by first creating a YAML configuration file like this:

```
(TDA) philc@colt:~/dataman$ cat transform.json
{ "input" : "/nfs/workspace/ge_data/census/household_tenure_const.csv",
  "output" : "/nfs/workspace/ge_data/census/new_hse_ten_const.csv",
  "primary_key" : "ONSConstID",
  "static_cols" : [ "ONSConstID", "ConstituencyName", "RegNationID",
```

```

        "RegNationName" ],
    "transform_cols" : [
        { "long_column" : "variables",
          "long_values" : "Con_num" },
        { "long_column" : "variables",
          "long_values" : "Con_pc" }
    ]
}

```

The input and output files are specified in the first two entries. The primary_{key} entry specifies how resulting DataFrames will be merged together based on the value of this field.

The static_{cols} key lists all the existing fields to be including in the resulting output file that are already in wide format. **Note that this list must include the primary_{key} field.**

Lastly, the transform_{cols} key provides a list of pairs of field names, the first of which identifies the field which contains the values that will be used to provide new column names in the output file, and the second identifies the field which contains the values to be used for these new column entries. In the case above, there are new wide columns to be provided for both constituency raw numbers and the constituency percentage of the uk population for the categories contained in the variables field.

Running the program is very simply a matter of executing long_{to}_wide.py with a suitable Python runtime system.

The end result looks like the listing below. Some column names have been changed from the default column names generated by the Pandas merge function.

cid	cname	regid	regname	soc_lndlrds_num	liv_rnt
-----	-----	-----	-----	-----	-----
E14001063	Aldershot	E12000008	South East	6,958	
0.045...	0.277...		0.355...		0.155...
E14001064	Aldridge-Brownhills	E12000005	West Midlands	5,260	
0.016...	0.420...		0.332...		0.091...
E14001065	Altrincham & Sale West	E12000002	North West	4,637	
0.015...	0.377...		0.365...		0.123...
E14001066	Amber Valley	E12000004	East Midlands	5,920	
0.019...	0.392...		0.307...		0.129...
E14001067	Arundel & South Downs	E12000008	South East	4,925	
0.027...	0.444...		0.286...		0.116...
E14001068	Ashfield	E12000004	East Midlands	6,320	
0.019...	0.359...		0.311...		0.150...
E14001069	Ashford	E12000008	South East	5,960	
0.022...	0.303...		0.345...		0.170...
E14001070	Ashton-under-Lyne	E12000002	North West	9,500	
0.020...	0.292...		0.284...		0.175...

E14001071 Aylesbury	E12000008 South East		6,833
0.019...	0.302...	0.361...	0.146..
E14001072 Banbury	E12000008 South East		6,402
0.024...	0.338...	0.300...	0.170..

Where is `longto__wide.py`

The Python source is located in a directory on **donkey.local**. You can access it in either of the following two ways:

- as "*nfs/workspace/CommonBirds/local/python/reuse/dataman*" if you are running Ubuntu inside a Windows WSL environment;
- as "*home/Public/workspace/CommonBirds/local/python/reuse/dataman*" if you are already logged into donkey.local (e.g. via ssh).

Please improve it!